

Article

An Evaluation of Modern Accelerator-Based Edge Devices for Object Detection Applications

Pilsung Kang ^{1,*}  and Athip Somtham ² ¹ Department of Software Science, Dankook University, Yongin 16890, Republic of Korea² Division of Computer Science and Engineering, Sunmoon University, Asan 31460, Republic of Korea

* Correspondence: pilsungk@dankook.ac.kr

Abstract: Edge AI is one of the newly emerged application domains where networked IoT (Internet of Things) devices are deployed to perform AI computations at the edge of the cloud environments. Today's edge devices are typically equipped with powerful accelerators within their architecture to efficiently process the vast amount of data generated in place. In this paper, we evaluate major state-of-the-art edge devices in the context of object detection, which is one of the principal applications of modern AI technology. For our evaluation study, we choose recent devices with different accelerators to compare performance behavior depending on different architectural characteristics. The accelerators studied in this work include the GPU and the edge version of the TPU, and these accelerators can be used to boost the performance of deep learning operations. By performing a set of major object detection neural network benchmarks on the devices and by analyzing their performance behavior, we assess the effectiveness and capability of the modern edge devices accelerated by a powerful parallel hardware. Based on the benchmark results in the perspectives of detection accuracy, inference latency, and energy efficiency, we provide a latest report of comparative evaluation for major modern edge devices in the context of the object detection application of the AI technology.

Keywords: edge computing; object detection; GPU (graphics processing unit); TPU (tensor processing unit); deep learning

MSC: 68M20

Citation: Kang, P.; Somtham, A. An Evaluation of Modern Accelerator-Based Edge Devices for Object Detection Applications. *Mathematics* **2022**, *10*, 4299. <https://doi.org/10.3390/math10224299>

Academic Editor: Freddy Gabbay

Received: 13 October 2022

Accepted: 15 November 2022

Published: 16 November 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

For the last decade, artificial intelligence (AI) technology has been widely applied for diverse problem domains and led to a variety of beneficial applications in our everyday lives. Among them, computer vision has grown to be one of the most essential domains due to its immediate applications to classifying images and detecting specific objects in complicated settings. Meanwhile, edge computing has recently emerged as an important paradigm as the amount of data produced at the IoT (Internet of Things) devices grows tremendously faster [1] and the bottlenecks within clouds due to network latency and bandwidth significantly degrade overall cloud performance [2]. Hence, the foremost premise of edge computing is efficient data processing and computation in place where data is generated from [3].

The concurrent rise of both the AI and the edge computing paradigms are commonly centered around the architectural developments of powerful hardware accelerators that assist the host CPU to deliver high performance in a small footprint. For instance, with its massive hardware parallelism and the support of programming models and toolchains surrounding the development environments, general purpose GPUs (graphics processing units) installed as a coprocessor of an edge device enable to realize complex AI applications that involve a significant amount of numerical computations even under harsh limitations of power consumption and memory capacity.

In this paper, we evaluate a set of latest edge devices for object detection applications. These modern edge devices usually feature different kinds of hardware accelerators for efficient numerical computation and data manipulation. Specifically, we use Google Coral Dev Board Mini (Coral Mini for short) [4], NVidia Jetson Nano Developer Kit (Jetson Nano) [5], NVidia Jetson Xavier NX Developer Kit (Jetson Xavier NX) [6] as our evaluation target devices. Unlike usual computing systems supported with a powerful CPU and a large amount of memory, these edge devices typically use embedded processors and a restricted amount of memory. However, all of these devices are equipped with specialized accelerators to offload parallel computations for boosting their performance for a certain type of AI applications. In particular, Google Coral Mini features the Edge TPU (tensor processing unit) designed for matrix operations in deep learning applications [7]. In contrast, NVidia Jetson devices are accelerated by the GPU coprocessor for performing general computing operations that can be utilized for deep learning applications as well.

To evaluate these modern edge devices in terms of AI applications, we perform a set of benchmarks based on deep neural networks. Especially, as we evaluated a set of entry-level edge devices using image classification CNNs (convolutional neural networks) in our previous work [8], we aim in this work to evaluate their performance and capabilities towards object detection, which is one of the principal application domains of AI technology. Using major object detection models as our evaluation tools, we examine and analyze the performance behavior based on three principal criteria: detection accuracy, inference latency, and energy efficiency. Through the evaluation, we search for proper perspectives for the applicability and usefulness of the edge devices for the object detection applications.

There have been many studies to evaluate edge devices in the context of AI applications [9–16]. However, most of them are getting gradually outdated or do not include latest devices with new hardware architectural features as the technology advances. In this paper, we aim to evaluate modern edge devices powered by different accelerators for object detection applications. Specifically, we make the following contributions:

- We perform a comparative analysis on latest accelerator-based edge devices with different architectural characteristics. These modern edge devices usually feature specialized hardware support for AI application domains such as computer vision. By choosing the latest offerings from the Google and the NVidia that provide different accelerator architectures, we compare the performance behavior and capabilities of the state-of-the-art edge devices in the context of the object detection applications.
- We categorize and report our evaluation results based on principle benchmark measures centered around the edge computing platforms towards the AI technology. In particular, our evaluation perspectives include object detection accuracy, performance latency, and energy efficiency in consideration of the edge computing environments. We also use well-known object detection models to better assess the capabilities of the targeted devices in edge computing.

Among other related research, our work is most similar to that of Hui et al. [9], where they share an early experience of evaluating three different kinds of edge AI processors with object detection workloads by means of a three-dimensional benchmarking methodology such as accuracy, latency, and energy efficiency. Although similar in terms of the performance metrics used in evaluation, our work is an up-to-date report which includes latest edge devices such as NVidia Jetson Nano and Xavier NX.

The remainder of this paper is structured as follows. Section 2 compares architectural differences of the evaluated edge devices, namely Google Coral Mini, NVidia Jetson Xavier NX, and Jetson Nano. The hardware and software setup information for the benchmarks is also provided. Section 3 describes performance criteria for evaluating the devices. Section 4 shows the benchmarks results of the evaluated devices, which are then analyzed according to the evaluation criteria. Section 5 contrasts our work with related research. Finally, Section 6 summarizes our work and make conclusions.

2. Setup: Hardware and Software for Benchmarks

In this section, we describe our overall evaluation setup. In particular, we present the hardware specifications of the evaluation target devices. We also present architectural features and software environments of these edge devices, where we carry out performance benchmark measurements. Then, we describe our evaluation criteria used in our evaluation in terms of the object detection application

2.1. Hardware Configuration

We evaluate three different edge device platforms which include NVidia Jetson Nano, Jetson Xavier NX, and Google Coral Dev Board Mini. All the devices are based on the system-on-module (SOM) connected to a development board, and commonly equipped with an accelerator for offloading performance critical workloads.

2.1.1. NVidia Jetson Nano

The Jetson Nano Developer Kit is a small but powerful edge device from NVidia. As one of the latest offerings from NVidia, Jetson Nano features a GPU coprocessor based on the Maxwell microarchitecture (GM20B). It comes with one streaming multiprocessor (SM) with 128 cores that can be used for parallel workloads including neural network calculations. Jetson Nano holds an on-board 4 GB of LPDDR4 DRAM which is shared by the CPU (Quad-core ARM Cortex-A57) and the GPU accelerator.

In terms of network connectivity, Jetson Nano includes only the Gigabit Ethernet to cut the production cost (MSRP \$99), which can be a drawback for adoption in wireless environments. Like other NVidia products, the CUDA (Compute Unified Device Architecture) programming model [17] is also supported for implementing parallel applications using the SIMT (single instruction, multiple threads) execution model for operating the cores of the GPU in parallel. In addition, a diverse set of CUDA programming libraries is also available, which includes cuDNN for deep learning [18] and cuBLAS for scientific computation [19].

2.1.2. NVidia Jetson Xavier NX

The Jetson Xavier NX is an advanced model specifically designed for AI applications. Within its very small footprint based on SOM, Jetson Xavier NX is equipped with a Volta microarchitecture-based GPU that comes with two SMs with 192 CUDA cores in each SM, which translates to $3\times$ more hardware parallelism than Jetson Nano. Furthermore, the GPU also boasts 48 tensor cores to boost the tensor operation performance of neural network applications.

The CPU of Jetson Xavier NX is also more powerful with its 6-core Carmel ARM architecture and 8 GB LPDDR4 DRAM memory can process data at close to 60 GB per second. On the whole, the Xavier NX module can deliver up to 21 TOPS (tera operations per second) for 8-bit integer operations at consuming only 15 Watts. On the software side, like other NVidia GPU products, Jetson Xavier NX is well supported by the CUDA programming ecosystem such as its toolchains and essential utility libraries like cuDNN.

2.1.3. Google Coral Dev Board Mini

Coral [20] is a hardware and software toolkit by Google for intelligent edge devices targeting AI applications on edge. Google Coral Dev Board Mini is one of the latest Coral devices as a single board computer in a small form factor that features the Edge TPU, a purpose-built integrated circuit for the edge environment. Like the GPU on the NVidia edge devices, the Edge TPU is utilized as a coprocessor for the Google Coral devices aimed at accelerating tensor operations in neural network applications on edge.

The Edge TPU is a small, low-power version of the TPU, which aggregates tens of thousands of ALUs (arithmetic logic units) using the systolic array architecture to pipeline the matrix multiplication operations over a wide range of multiply accumulate (MAC) units. In this architecture, each multiplication result is passed to next MACs that can

perform multiplication and summation at the same time. However, since the Edge TPU is only specialized for tensor operations, it cannot be effectively used for a wide range of general computations across different applications as supported by the GPU of the NVidia products.

Coral Mini is equipped with 2 GB memory, which is much smaller compared to the NVidia Jetson devices, but is $2\times$ bigger than its predecessor Google Coral Dev Board. However, the Mini version uses the DDR3 DRAM instead of DDR4 as used in the regular version, possibly due to production cost reduction. In addition to the change in memory, Coral Mini uses Quad-core ARM Cortex-A35 as its CPU, which is 25% smaller compared to the Cortex-A53 core used in the regular Coral Dev Board version. ARM Cortex-A35 is a newer generation than Cortex-A53 and has been announced to deliver more performance-per-watt efficiency. Specifically, ARM announced that Cortex-A35 consumes 32% less power and is 25% more efficient than Cortex-A53 [21].

Table 1 compares the specification of the three edge devices. In summary, the NVidia Jetson Nano and Xavier NX devices are commonly equipped with the GPU accelerator for processing parallel workloads in general, which can be effectively used for deep learning operations where numerical calculations with vectors and matrices can be performed in parallel. In contrast, Coral Mini utilizes Edge TPU, a purpose-built ASIC (application-specific integrated circuit) for calculation and data analysis based on matrix operations in AI applications. Because of its limited versatility compared to the GPU of the NVidia devices, it is used as a coprocessor to assist the CPU in performing deep learning tasks. However, the tensor operations can be processed fairly quickly, and the power consumption is also quite low, so it is expected to be adequately suitable for use in object detection applications at the edge of the cloud.

Table 1. Specifications of NVidia Jetson Nano, NVidia Jetson Xavier NX, and Google Coral Mini.

	Jetson Nano	Jetson Xavier NX	Coral Mini
Processor	Quad-core ARM Cortex-A57	Hexa-core Carmel ARM v8.2 CPU	Quad-core ARM Cortex-A35
Accelerator	GPU (128 CUDA cores, 472 GFLOPs)	GPU (384 CUDA cores, 48 Tensor cores, 21 TOPs)	Edge TPU (Systolic array, 4 TOPs)
Memory	4 GB LPDDR4	8 GB LPDDR4x	2 GB LPDDR3
Flash Memory	16 GB eMMC	16 GB eMMC	8 GB eMMC
Supported Frameworks	Major ML Frameworks (TensorFlow, PyTorch, and Caffe)		TensorFlow Lite
Networking	10/100/1000 BASE-T Ethernet		Wi-Fi 5, Bluetooth 5.0
TDP	5~10 W	10~20 W	12.5~15 W
Cost	\$99	\$399	\$99.99

On the software side, NVidia Jetson devices support different programming libraries including TensorRT, which supports different deep learning frameworks such as PyTorch, Caffe, and TensorFlow. By contrast, Google Coral Dev Board Mini supports only TensorFlow Lite, which requires that artificial neural network models need to be trained in consideration of quantization [22]. In addition, only 8-bit integers are supported for representing network model parameters.

2.2. Deep Learning Libraries and Models

To perform object detection benchmarks for the targeted devices, we use two popular neural network models: YOLOv4-Tiny and SSD MobileNet V2. For the dataset of the objection detection evaluation, we use Microsoft Common Objects in Context [23] (MS COCO), which is a large-scale dataset for object detection, segmentation, key-point detection, and captioning applications. The MS COCO dataset consists of hundreds of thousands

of images of everyday objects and humans, together with annotations arranged in the JSON format, which can be used to train machine learning models to recognize and label objects.

2.2.1. YOLOv4-Tiny

YOLO (You Only Look Once) is a real-time object detection model for detecting multiple objects in a single frame [24]. YOLO's detection algorithm is known to achieve high accuracy even with its single forward propagation pass through the neural network. With YOLO, multiple bounding boxes and class probabilities for detected objects can be simultaneously found. YOLOv4-Tiny is a compressed version of YOLOv4, but with a simpler network structure and reduced parameters, thereby making it more favorable for development of AI applications on mobile and embedded devices. The number of model parameters of YOLOv4-Tiny is about 4 million and the model size is 16 MB (float32). Regarding its detection accuracy, YOLOv4-Tiny is known to achieve 40 mAP (mean average precision) on the MS COCO dataset at IoU threshold 0.5 (see Section 3 for the accuracy metric descriptions).

2.2.2. SSD MobileNet V2

MobileNet [25] is a small model with low latency and computing power commonly deployed on low compute devices such as mobile phones. Designed for resource-constrained tasks, MobileNet can be used in classification, detection, embedding, and segmentation like other popular models such as ResNet and Inception [26]. MobileNet can often be less accurate than other popular full-featured models. However, it comes with less latency and a much smaller model size, which allows for faster processing on embedded systems.

The SSD (Single Shot Detector) object detection is a single stage algorithm that consists of feature map extraction and convolution filter application to detect objects. Based on the SSD algorithm, SSD MobileNet V2 uses MobileNet as its backbone and adopts novel depthwise separable convolutions to realize optimized detection performance on mobile devices [27]. The number of model parameters of SSD MobileNet V2 is 15.3 million and the model size is about 63 MB (float32).

2.3. Evaluation Setup

We proceed with a thousand of images in the COCO dataset on the evaluation target edge devices. Each model performs object detection to generate detection results in a JSON file containing image IDs, category IDs, and detection scores information. The JSON output file can be submitted to the MS COCO evaluation server (COCO: Upload Results to Evaluation Server, <https://cocodataset.org/#upload>, accessed on 9 October 2022) to calculate the detection accuracy.

2.3.1. NVidia Jetson Nano and Xavier NX Setup

We use pre-trained deep learning models that are publicly available [28,29]. The pre-trained models cover image classification, object detection, semantic segmentation, and pose estimation. Figure 1 illustrates the workflow on NVidia Jetson devices. The network model is built by first parsing the convolutional neural network architecture (CNN) specification file given as input. Then, the network is constructed by using the model parameters defined in the *prototxt* (plaintext protocol buffer schema) format. Next, the TensorRT library compiles and optimizes the pre-trained model for the execution environment, thus completing the model build process. Overall, Jetson devices can perform inference by utilizing the deep learning frameworks, a *prototxt* file representing the network model structure, and the TensorRT engine.

In NVidia Jetson devices, we used the JetPack SDK (software development kit) v4.3 to flash the storage on Linux for Tegra 32.2.2, along with other supporting libraries for CUDA like cuDNN, for deploying the object detection benchmarks.

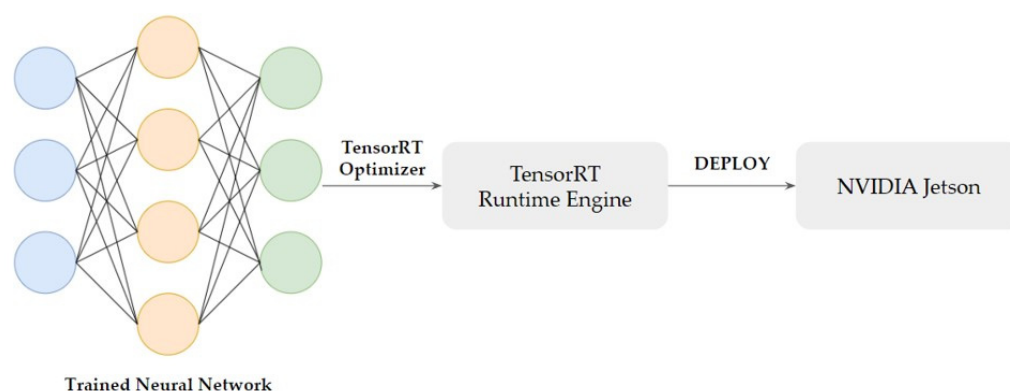


Figure 1. Deep Learning Model Deployment Workflow on NVidia Jetson Devices.

Specifically, to measure the accuracy and latency, we use the pre-trained YOLOv4-Tiny and SSD MobileNet v2 models and benchmark scripts from [28,29]. For the test data, we randomly choose a thousand images from the MS COCO ‘test-dev2017’ dataset. Running the benchmark script for each image generates latency information and detection results in the JSON format, which can be zipped to submit to the MS COCO evaluation server. To measure energy efficiency, we connect a power meter to each device and measure average power consumption while running the detection models. The number of processed images, benchmark running time, and the measured power consumption numbers give energy efficiency.

2.3.2. Google Coral Dev Board Mini Setup

The workflow to deploy a neural network model on the Edge TPU is shown in Figure 2. In general, the Coral environment uses TensorFlow [30] to define and learn the structure of the CNN model. In order to apply a TensorFlow model to Coral edge devices, the trained model structure is required to be turned into a TensorFlow Lite (tflite) model through the quantization process, where the model parameters are represented as 8-bit integers. Next, the model is compiled by the Edge TPU compiler for execution on the Edge TPU runtime. Here, the compiler separates out the operations supported by the Edge TPU, whereas the other operations are separately executed on the CPU. After compiling for Edge TPU, the inference result is then obtained via a runtime within the Coral device. As in the NVidia Jetson setup, we use publicly available inference models for Coral Mini [31,32]. Except that the detection models need to be recompiled to work on the Edge TPU, the steps to measure latency, accuracy, and energy efficiency are similar to the steps for NVidia Jetson devices.

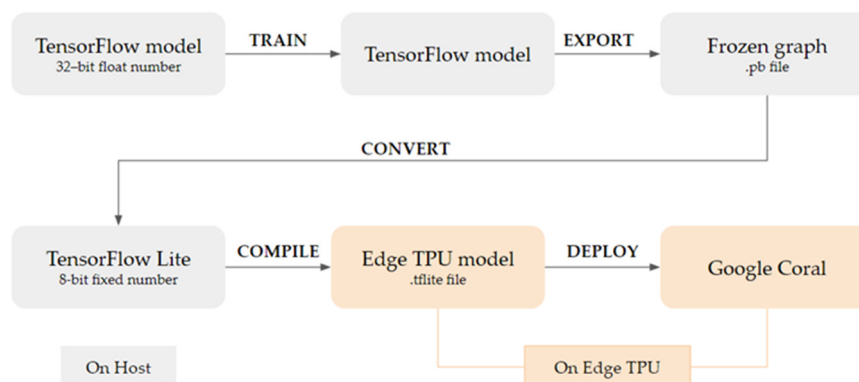


Figure 2. Deep Learning Model Deployment Workflow on Google Coral Dev Board.

One thing to note is that the YOLOv4-Tiny model uses leaky ReLU as its activation function, which has a small slope for negative values instead of a flat slope as in the regular ReLU function. However, leaky ReLU is not supported on the Edge TPU [33], so the model

needs to be transformed to use the regular ReLU in the conversion step for proper execution on the Edge TPU. The replacement of the leaky ReLU with the regular ReLU can affect the Edge TPU's performance behavior in a negative way as we discuss later in Section 4.

3. Evaluation Perspectives

We benchmark and analyze the edge devices according to three-dimensional evaluation perspectives as suggested in [9]. The evaluation principles we apply to the benchmark results are described in the following.

- **Accuracy:** In object detection problems the mostly used metric is average precision (AP). AP can be calculated by Equation (1). For the COCO dataset, AP is averaged over all categories of the images to give mean AP (mAP). Equation (2) shows the definition of mAP in the mathematical form, where N is the total number of categories. For instance, the MS COCO dataset contains 80 categories.

Average Precision:

$$AP = \frac{1}{11} \sum_{r \in \{0,0.1,\dots,1.0\}} P_r \quad (1)$$

where P_r is the interpolated precision value and r corresponds to the recall value.

Mean Average Precision

$$mAP = \frac{\sum_{i=1}^N AP}{N} \quad (2)$$

When measuring the detection accuracy in mAP, we increase the IoU (intersection over union) threshold by 0.05 from 0.5 to 0.95, and then we calculate the mAP using the accuracy of each step. The accuracy measurements vary depending on the preset IoU threshold value. In Figure 3, we show how the detection results vary depending on different IoU threshold values. In the figure, the threshold is set to 0 in the left and 0.5 in the right. As shown in the figure, when the threshold is too small, the detector becomes too sensitive (left), and it is important to find and decide an appropriate value for the IoU threshold.

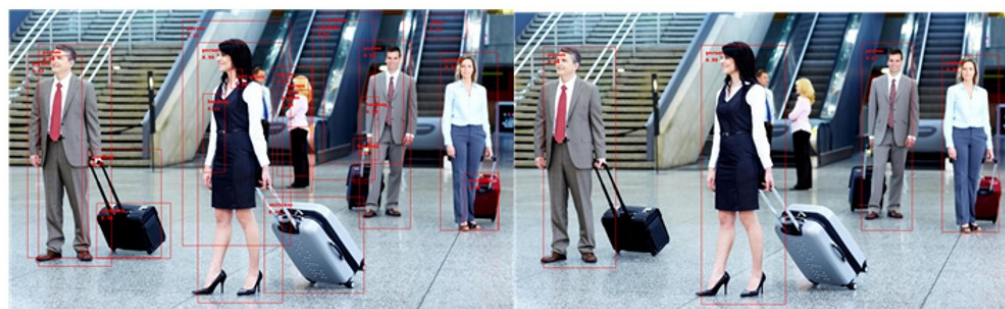


Figure 3. Change of Detection Accuracy depending on the IoU threshold. Threshold is 0 (left) and 0.5 (right).

One thing to note regarding detection accuracy is that it is a metric rather for the CNN model itself than for the edge device used. However, we include the accuracy in evaluating the devices because of the architectural differences between Jetson devices and Coral Mini which can affect the detection accuracy. For example, CNN model parameters are usually represented with 32-bit floating point numbers. However, the Edge TPU of Coral Mini uses 8-bit quantized integer parameters, which allows for savings in the memory usage but with potential losses in precision.

- **Latency:** As the most important performance metric in AI applications, latency is defined to be the time to complete the inference process for one batch of input images. For object detection applications on embedded systems, it is critical that the latency should be maintained as low as possible since edge applications are usually operated at real-time. As to the batch size, it is typical that the batch size for edge device applications is set to a small value mainly due to the limited computing power and

resources of the device such as memory capacity. In our evaluation, we set the batch size to be 1.

- **Energy Efficiency:** This performance metric is used to assess how well the edge device performs in terms of consumed energy, and in our evaluation setting, it is expressed as the number images processed per watt in a unit time. Since the edge environment is usually strict in terms of power consumption, this metric is also a critical factor in evaluating edge devices.

4. Evaluation Results and Analysis

We evaluate Google Coral Dev Board Mini and NVidia Jetson devices based on the performance behavior across the object detection CNN benchmark models used for benchmark. As we previously mentioned, we set the batch size to 1 at the inference stage, since processing multiple images is typically not very profitable and considered as ineffective on the edge computing environments.

The measurements are performed in the text mode of the system, so as to minimize the use of computing resources without using the graphics mode. NVidia Jetson devices may show degraded performance when using the graphics mode because they use the GPUs for acceleration.

4.1. Detection Accuracy

Figure 4 shows the measurement results of the detection accuracy in mAP between Jetson Nano, Jetson Xavier NX, and Coral Mini with YOLOv4-Tiny and SSD MobileNet v2 on the MS COCO dataset. We observe that Jetson Xavier NX performs the best with the mAP value of 0.29 for both detection models. On the contrary, Coral Dev Board Mini performs the worst compared to other devices. This is in contrast to the observations in our previous work [8], where Coral Dev Board, even with the 8-bit quantization scheme of the Edge TPU, showed comparable inference accuracy without resulting in substantial degradation in inference quality when tested with image classification CNNs. Jetson Nano's detection performance is placed in the middle of Jetson Xavier NX and Coral Dev Board Mini.

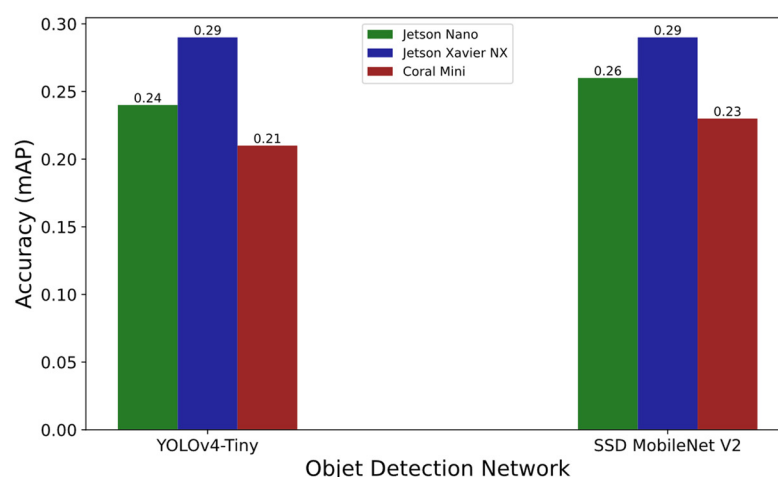


Figure 4. Detection Accuracy Measurement Results of the Devices.

Beside the 8-bit quantization, we suspect that the detection accuracy of Coral Mini can be harmed by the use of the regular ReLU as the activation function. As we mentioned before, the Edge TPU does not support the leaky ReLU operation of YOLOv4-Tiny and the conversion step had to replace leaky ReLU with the regular ReLU when preparing the deployment model for the Edge TPU. It has been reported that the regular ReLU activation function performs 5 to 10% worse compared to other variants of ReLU including the leaky ReLU [34]. Typically, the inference accuracy is mostly determined by how the given neural

network model is structured and constructed. However, interestingly, we observe here that the performance behavior of a model can be affected by the runtime environment configuration such as the execution hardware as in this case.

4.2. Latency

Figure 5 compares the inference latency of the used detection network models between the Jetson devices and Coral Mini. Jetson Xavier NX shows the best performance here for both the network models. For instance, with regard to the YOLOv4-Tiny model, Jetson Xavier NX performs 75% faster than Jetson Nano, and 79% faster than Coral Mini. Since Jetson Xavier NX is equipped with the most powerful hardware resources including the host CPU (6 cores), the GPU accelerator (384 CUDA cores), and the memory (8 GB), the observed best-performing behavior among the examined devices can be considered very natural.

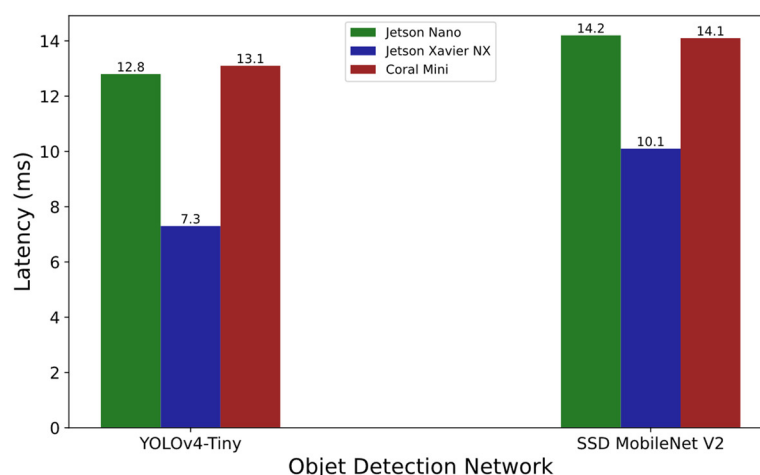


Figure 5. Latency Measurement Results of the Devices.

However, considering that the cost of Jetson Xavier NX is $4\times$ more expensive than other devices, it is not performing very attractive because it implies to show more than 50% worse behavior in terms of the performance per cost ratio. In addition to the performance-per-cost perspective, the performance-per-watt behavior of the tested devices is discussed later in the next subsection.

Coral Mini and Jetson Nano show almost the same latency results for both YOLOv4-Tiny and SSD MobileNet V4. We observe that the SRAM caching effect of the Edge TPU is not dominant in our experiments. In our previous work [8], the Google Coral device showed up to $5\times$ better performance than Jetson Nano for simple image classification networks thanks to the 8 MB of SRAM cache of the Edge TPU.

Typically, some portion of the SRAM is allocated by the Edge TPU Compiler to cache the model parameters. Simple neural networks with small-sized parameters can show substantial performance improvements on the Edge TPU thanks to the Edge TPU SRAM caching. However, the object detection networks used in our evaluation have a larger amount of parameters than what the SRAM of the Edge TPU can comfortably accommodate, thereby eliminating the opportunity for potential performance boost due to caching.

4.3. Energy Efficiency

Figure 6 compares the examined devices according to the energy efficiency perspective in executing each object detection network model. The results show that Jetson Nano performs the worst in processing images per unit energy for both YOLOv4-Tiny and SSD MobileNet V2.

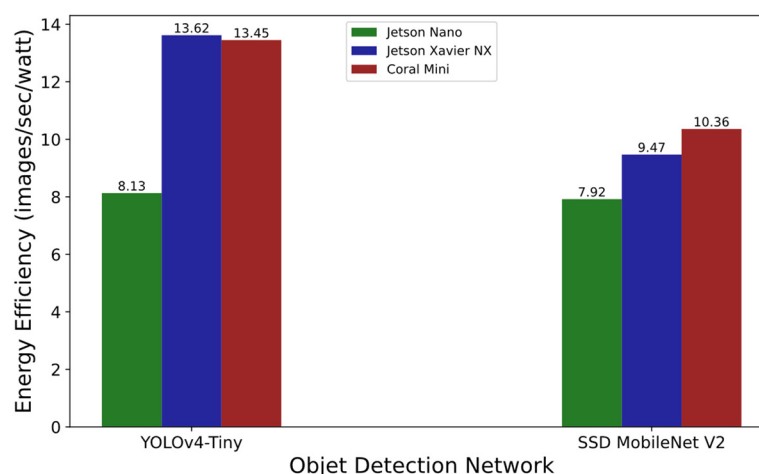


Figure 6. Energy Efficiency Measurement Results of the Devices.

Jetson Xavier NX and Coral Mini show somewhat different performance behavior depending on the used object detection network. For the YOLOv4-Tiny model, Jetson Xavier NX shows the best energy efficiency. It performs 67.5% better than Jetson Nano and 1% better than Coral Mini. Considering measurement deviations, Jetson Xavier NX and Coral Mini show comparable performance each other. For the SSD MobileNet V2 model, it performs 20% better than Jetson Nano, which is significantly lower compared to the results for YOLOv4-Tiny. This is assumed to be due to the difference in the model complexity between the two network models. For the relatively simpler YOLOv4-Tiny model, the inference speed varies bigger among the tested devices. By contrast, for the more complex SSD MobileNet V2 model, the inference step becomes slower for the devices on the whole and it varies relatively smaller across the devices.

Overall, Coral Mini shows the most favorable energy efficiency behavior. For YOLOv4-Tiny, its efficiency is observed to be similar to the efficiency of Jetson Xavier NX which is more powerful in terms of hardware resources. Compared to Jetson Nano, Coral Mini shows 65% better energy efficiency.

In addition, it shows the best performance-per-watt for the SSD MobileNet V2 model. Coral Mini performs 9% better than Jetson Xavier NX and 31% better than Jetson Nano. Considering Coral Mini's price is almost the same as that of Jetson Nano, Coral Mini's performance per power behavior can be considered quite attractive in this application domain. In fact, as shown in Table 1, Jetson Nano's TDP is smaller than that of Coral Mini, which makes Jetson Nano more favorable if we only consider this metric. However, if we consider performance per watt (i.e., number of processed images per second per watt), our measurements show Coral Mini performs better. We note that this result is based on only object detection applications, and the performance behavior may vary depending on particular applications.

In our previous work [8], we observed and reported that Coral Dev Board used about 10% less power than Jetson Nano for a diverse set of image classification networks. Now, the Coral device shows substantial improvements in the performance-per-watt behavior, which amounts to 47% better efficiency than Jetson Nano on average. Although the experimental results are drawn from only two object detection networks, Coral Mini's power efficiency results are very attractive.

Table 2 summarizes our benchmark measurement results for both YOLOv4-Tiny and SSD MobileNet V2 across Jetson Nano, Jetson Xavier NX, and Coral Mini.

Table 2. Performance Measurement Results of Each Device for Object Detection Networks.

Detection Network	Performance Metric	Jetson Nano	Jetson Xavier NX	Coral Mini
YOLOv4-Tiny	Accuracy (mAP)	0.24	0.29	0.21
	Latency (ms)	12.8	7.3	13.1
	Energy Efficiency (images/sec/watt)	8.13	13.62	13.45
SSD	Accuracy (mAP)	0.26	0.29	0.23
MobileNet V2	Latency (ms)	14.2	10.1	14.1
	Energy Efficiency (images/sec/watt)	7.92	9.47	10.36

5. Related Work

There is a sizable amount of research on benchmark techniques, tools, and evaluation reports on edge computing, and we briefly survey most remarkable research works in this section. As to the performance benchmarking in general on the edge computing environment, Varghese et al. [10] present a detailed survey on edge performance benchmarking, which summarizes various benchmark techniques and quality metrics surrounding the edge computing paradigm. For a comprehensive survey on diverse accelerator architectures for deep learning applications, we refer the reader to [11] by Chen et al.

Choosing an appropriate platform and the right inference model becomes a challenging task for the machine learning engineers as use cases and deployment scenarios diversify. Schneider et al. develop a universal approach to this task by implementing a scalable benchmark architecture with a container-based benchmark runtime [12]. By doing so, they provide a unified benchmark architecture for evaluating edge devices, allowing to examine optimal operating points for different deployment scenarios.

There are many studies and evaluation reports on recent Edge devices, and we briefly describe some of the most notable ones here. Allan [13] compares inference performance on Google Coral Dev Board and NVidia Jetson Nano using the SSD MobileNet V1 and V2 models trained on the COCO dataset, but the experiment is only focused on latency measurements. Antonini et al. [14] evaluate and compare Google Coral Dev Board, NVidia Jetson Nano, Intel Neural Compute Stick using CNN benchmarks. In contrast, our work is focused on AI performance evaluation for object detection applications with more recent and powerful edge devices including Jetson Xavier NX.

Feng et al. [15] analyze the performance of four different versions of YOLO on three latest edge devices including NVidia Jetson Nano, NVidia Jetson Xavier NX, and Raspberry Pi 4B, and report that Jetson Nano is the most favorable among the three devices considering in terms of performance per price. Their work is most similar to ours in that latest devices are evaluated. However, the most significant difference of our work compared to theirs is that our work focuses on the comparative evaluation of modern accelerator-based edge devices, which are primarily the GPUs and TPUs. While their work also studies NVidia GPU based devices, their evaluation does not cover the edge TPU, which is the most popular accelerator in edge computing nowadays. In addition, their evaluation is only focused on the YOLO performance, thereby lacking the performance behavior assessment of different object detection network models on the tested edge devices.

Baller et al. [16] present a comparative evaluation of Raspberry Pi 4, Google Coral Dev Board, NVidia Jetson Nano, and Arduino Nano 33 BLE, using different deep learning models. They report that the Google device performs best for TensorFlow models in terms of inference time and power consumption. Their work is also similar to ours, but is in contrast to ours where NVidia Jetson Xavier NX is included for evaluation for comparison with other recent edge devices.

Other than the GPU and TPU based accelerators, there are other architectural approaches to realize edge AI under highly power-constraint environments. GAP processors [35] are based on the RISC-V open standard instruction set architecture and can perform up to 22 GOPS under 100 mW, which is much lower than our tested devices. AI application programming support is also getting expanded [36]. On the other hand, FPGA (field-programmable gate array) based solutions are also emerging due to their flexible customization capability as well as low-power consumption behavior [37]. However, more thorough and detailed evaluation studies are essential to a more widespread adoption of these architectures.

6. Summary and Conclusions

In this paper, we evaluated three major modern edge devices—Google Coral Dev Board Mini, NVidia Jetson Nano, and NVidia Jetson Xavier NX in terms of the object detection capability in AI applications. These state-of-the-art devices are equipped with different types of accelerator architecture as a coprocessor, which can be used to accelerate AI applications. Using a set of object detection models such as YOLOv4-Tiny and SSD MobileNet V2 to measure their detection accuracy, performance latency, and energy efficiency, we evaluated the capability of each device.

Jetson Xavier NX shows the most favorable behavior across all the performance metrics. In particular, it provides the best latency behavior thanks to its powerful SIMT-based GPU accelerator, thus making it well suited to performance-critical applications. However, it is $4\times$ more costly than the other devices, which can be a drawback for widespread adoption compared to other devices. Coral Dev Board Mini shows very attractive power-efficiency behavior mainly due to its power-centric design across the module, which includes the use of the Cortex-A35 CPU, the Edge TPU accelerator, and small footprint. For instance, it shows 47% better energy efficiency on average compared to Jetson Nano for the object detection inference workloads. The use of Coral Dev Board Mini can significantly benefit such deep learning applications that need to operate under low-power environments. However, the inference accuracy requirement needs to be carefully considered when using the Edge TPU accelerator. As we discussed in our evaluation, the Edge TPU requires 8-bit quantization of model parameters and it does not support certain popular deep learning operations, which can result in degraded accuracy depending on the used neural network. Jetson Nano shows an adequate performance overall. It is placed in the middle in terms of detection accuracy and its latency behavior is comparable to Coral Mini. However, its performance-per-watt result is least favorable among the examined devices. Considering its low cost and applicability to general purpose applications, Jetson Nano with its GPU can be a viable option for a wide range of AI applications without too strict power consumption requirements.

As the edge AI technology continues to mature, the hardware gets more and more powerful with greater parallelism or with novel architectural features. In this regard, we plan to continue to evaluate latest edge devices such as Jetson Orin [38], a new family of the NVidia Jetson series, as they become available in the future. In addition, we plan to further our edge device evaluations using different kinds of tasks, where we expect a very different trends in performance behavior across the devices depending on the workloads. For natural language processing applications as an example, the sequential nature of recurrent neural networks (RNNs) requires specialized architectural features such as optimized data paths and control for efficient processing. This is in contrast with a large amount of hardware parallelism typically adopted in accelerators for boosting CNN performance in image classification and object detection applications.

We also plan to examine the power behavior of edge devices in different settings such as using a varied number of detection categories. In real time scenarios for example, the detection system may have to detect only moving objects like people without having to recognize all the categories in the training dataset. This would significantly simplify the inference process and thereby improve the power behavior of the edge device. Therefore, a

quantitative evaluation of such effects will be an interesting exploration. In conclusion, as the edge AI applications expand, a more refined and multi-faceted methodology will be essential in evaluating modern edge devices.

Author Contributions: Conceptualization, P.K.; methodology, P.K.; software, P.K.; validation, P.K. and A.S.; formal analysis, P.K.; investigation, P.K. and A.S.; resources, P.K.; data curation, P.K. and A.S.; writing—original draft preparation, P.K. and A.S.; writing—review and editing, P.K. and A.S.; visualization, P.K. and A.S.; supervision, P.K.; project administration, P.K.; funding acquisition, P.K. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT), grant number 2020R1F1A1067619. The APC was funded by 2020R1F1A1067619.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The MS COCO datasets used in our experiments are publicly available at <https://cocodataset.org> (accessed on 9 October 2022).

Conflicts of Interest: The authors declare no conflict of interest.

References

- Reinsel, D.; Gantz, J.; Rydning, J. The Digitization of the World from Edge to Core, IDC White Paper, November 2018. Available online: <https://www.seagate.com/files/www-content/our-story/trends/files/idc-seagate-dataage-whitepaper.pdf> (accessed on 9 October 2022).
- Varghese, B.; Wang, N.; Barbhuiya, S.; Kilpatrick, P.; Nikolopoulos, D.S. Challenges and opportunities in edge computing. In Proceedings of the IEEE International Conference on Smart Cloud, New York, NY, USA, 18–20 November 2016; pp. 20–26.
- Shi, W.; Cao, J.; Zhang, Q.; Li, Y.; Xu, L. Edge Computing: Vision and Challenges. *IEEE Internet Things J.* **2016**, *3*, 637–646. [CrossRef]
- Google Coral Dev Board Mini. Available online: <https://coral.ai/products/dev-board-mini> (accessed on 9 October 2022).
- NVIDIA Jetson Nano Developer Kit. Available online: <https://developer.nvidia.com/embedded/jetson-nano-developer-kit> (accessed on 9 October 2022).
- NVIDIA Jetson Xavier NX. Available online: <https://developer.nvidia.com/embedded/jetson-xavier-nx> (accessed on 9 October 2022).
- Jouppi, N.P.; Young, C.; Patil, N.; Patterson, D.; Bajwa, R.; Bates, S.; Bhatia, S.; Boden, N.; Borchers, A.; Boyle, R.; et al. In-Datcenter Performance Analysis of a Tensor Processing Unit. In Proceedings of the 44th Annual International Symposium on Computer Architecture, Toronto, ON, Canada, 24–28 June 2017; pp. 1–12.
- Kang, P.; Jo, J. Benchmarking Modern Edge Devices for AI Applications. *IEICE Trans. Inf. Syst.* **2021**, *E104D*, 394–403. [CrossRef]
- Hui, Y.; Lien, J.; Lu, X. Early Experience in Benchmarking Edge AI Processors with Object Detection Workloads. *Lect. Notes Comput. Sci.* **2020**, *12093*, 32–48. [CrossRef]
- Varghese, B.; Wang, N.; Bermbach, D.; Hong, C.H.; Lara, E.D.; Shi, W.; Stewart, C. A Survey on Edge Performance Benchmarking. *ACM Comput. Surv.* **2021**, *54*, 1–33. [CrossRef]
- Chen, Y.; Xie, Y.; Song, L.; Chen, F.; Tang, T. A Survey of Accelerator Architectures for Deep Neural Networks. *Engineering* **2020**, *6*, 264–274. [CrossRef]
- Schneider, M.; Prokscha, R.; Saadani, S.; Höß, A. ECBA-MLI: Edge computing benchmark architecture for machine learning inference. In Proceedings of the 2022 IEEE International Conference on Edge Computing and Communications (EDGE), Barcelona, Spain, 11–15 July 2022; pp. 23–32.
- Allan, A. Benchmarking Edge Computing. Available online: <https://aallan.medium.com/benchmarking-edge-computing-ce3f13942245> (accessed on 9 October 2022).
- Antonini, M.; Vu, T.H.; Min, C.; Montanari, A.; Mathur, A.; Kawsar, F. Resource characterisation of personal-scale sensing. Models on Edge Accelerators. In Proceedings of the First International Workshop on Challenges in Artificial Intelligence and Machine Learning for Internet of Things, New York, NY, USA, 10–13 November 2019; pp. 49–55.
- Feng, H.; Mu, G.; Zhong, S.; Zhang, P.; Yuan, T. Benchmark Analysis of YOLO Performance on Edge Intelligence Devices. *Cryptography* **2022**, *6*, 1–16. [CrossRef]
- Baller, S.P.; Jindal, A.; Chadha, M.; Gerndt, M. DeepEdgeBench: Benchmarking deep neural networks on edge devices. In Proceedings of the 2021 IEEE International Conference on Cloud Engineering (IC2E), San Francisco, CA, USA, 4–8 October 2021; pp. 20–30.
- Nickolls, J.; Buck, I.; Garland, M.; Skadron, K. Scalable Parallel Programming with CUDA. *ACM Queue* **2008**, *6*, 40–53. [CrossRef]
- Chetlur, S.; Woolley, C.; Vandermersch, P.; Cohen, J.; Tran, J.; Catanzaro, B.; Shelhamer, E. Cudnn: Efficient Primitives for Deep Learning. *arXiv* **2014**, arXiv:1410.0759.

19. The CUDA Basic Linear Algebra Subroutine Library. Available online: <https://docs.nvidia.com/cuda/cublas> (accessed on 9 October 2022).
20. Google Coral. Available online: <https://coral.ai> (accessed on 9 October 2022).
21. Frumusanu, A. ARM Announces New Cortex-A35 CPU—Ultra-High Efficiency for Wearables & More. 2015. Available online: <https://www.anandtech.com/show/9769/arm-announces-cortex-a35> (accessed on 9 October 2022).
22. Jacob, B.; Kligys, S.; Chen, B.; Zhu, M.; Tang, M.; Howard, A.; Hartwig, A.; Kalenichenko, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–22 June 2018; pp. 2704–2713. [CrossRef]
23. Lin, T.Y.; Maire, M.; Belongie, S.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; Zitnick, C.L. Microsoft COCO: Common Objects in Context. In Proceedings of the European Conference on Computer Vision, Zurich, Switzerland, 6–12 September 2014; Springer: Cham, Switzerland, 2014.
24. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 26 June–1 July 2016; pp. 779–788. [CrossRef]
25. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Adam, H. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv* **2017**, arXiv:1704.04861.
26. Szegedy, C.; Ioffe, S.; Vanhoucke, V.; Alemi, A.A. Inception-V4, inception-resnet and the impact of residual connections on learning. In Proceedings of the 31st AAAI Conference on Artificial Intelligence, San Francisco, CA, USA, 4–9 February 2017.
27. Sandler, M.; Howard, A.; Zhu, M.; Zhmoginov, A.; Chen, L.C. MobileNetV2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–22 June 2018.
28. Jetson Inference Models. Available online: <https://github.com/dusty-nv/jetson-inference> (accessed on 9 October 2022).
29. Darknet: Open Source Neural Networks in C. Available online: <https://github.com/AlexeyAB/darknet> (accessed on 9 October 2022).
30. Abadi, M.; Barham, P.; Chen, J.; Chen, Z.; Davis, A.; Dean, J.; Devin, M.; Ghemawat, S.; Irving, G.; Isard, M.; et al. TensorFlow: A system for large-scale machine learning. In Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), Savannah, GA, USA, 2–4 November 2016; pp. 265–283.
31. YOLOv4 for TensorFlow. Available online: <https://github.com/hhk7734/tensorflow-yolov4> (accessed on 9 October 2022).
32. PyCoral API. Available online: <https://github.com/google-coral/pycoral> (accessed on 9 October 2022).
33. TensorFlow Models on the Edge TPU on Coral. Available online: <https://coral.ai/docs/edgetpu/models-intro/#supported-operations> (accessed on 9 October 2022).
34. Xu, B.; Wang, N.; Chen, T.; Li, M. Empirical Evaluation of Rectified Activations in Convolutional Network. *arXiv* **2015**, arXiv:1505.00853.
35. Flamandm, E.; Rossi, D.; Conti, F.; Loi, I.; Pullini, A.; Rotenberg, F.; Benini, L. GAP-8: A RISC-V SoC for AI at the edge of the IoT. In Proceedings of the IEEE 29th International Conference on Application-Specific Systems, Architectures and Processors (ASAP), Milan, Italy, 10–12 July 2018; pp. 1–4.
36. Garofalo, A.; Rusci, M.; Conti, F.; Rossi, D.; Benini, L. PULP-NN: A Computing library for quantized neural network inference at the edge on RISC-V based parallel ultra low power clusters. In Proceedings of the 26th IEEE International Conference on Electronics, Circuits and Systems (ICECS), Genoa, Italy, 27–29 November 2019; pp. 33–36.
37. Biookaghazadeh, S.; Zhao, M.; Ren, F. Are FPGAs suitable for edge computing? In Proceedings of the USENIX Workshop on Hot Topics in Edge Computing (HotEdge 18), Boston, MA, USA, 10 July 2018.
38. NVidia Jetson Orin Modules and Developer Kit. Available online: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin> (accessed on 9 October 2022).